Conditional Statements in Python:

In Python, condition statements act depending on whether a given condition is true or false. You can execute different blocks of codes depending on the outcome of a condition. Condition statements always evaluate to either True or False.

1. Indentation:

In Python, indentation is crucial for indicating blocks of code. Indentation is used to define the scope of a block of code, and it is a fundamental aspect of Python's syntax.

```
Syntax:
if condition:
    # indented block of code
    # executed when the condition is true
```

2. The If Statement and its Related Statement:

The `if` statement is the simplest form. It takes a condition and evaluates to either `True` or `False`.

If the condition is `True`, then the True block of code will be executed, and if the condition is False, then the block of code is skipped, and The controller moves to the next line

```
Syntax:
if condition:
     statement 1
     statement 2
     statement n

#Example:
number = 6
if number > 5:
    # Calculate square
```

```
    print(number * number)
print('Next lines of code')
```

3. An Example with If and its Related Statement:

```
# Voting Eligibility Checker

# Get the age of the person
age = int(input("Enter your age: "))

# Define the minimum voting age
voting_age = 18

# Check eligibility using if statement
if age >= voting_age:
    # indented block of code
    # executed when the person is eligible to vote
    print("You are eligible to vote. Exercise your right to v
else:
    # indented block of code
    # executed when the person is not eligible to vote
    years_until_eligible = voting_age - age
    print(f"Sorry, you are not eligible to vote yet. You can
```

4. Else:

The `else` statement is used to provide an alternative block of code to execute when the condition in the `if` statement is false.

```
Syntax:
if condition:
    # code block to execute if condition is true
else:
    # code block to execute if condition is false
Example:
password = input('Enter password ')

if password == "PYnative@#29":
```

```
    print("Correct password")
else:
    print("Incorrect Password")
```

5. if-elif-else:

the `if-elif-else` condition statement has an `elif` blocks to chain multiple conditions one after another. This is useful when you need to check multiple conditions.

With the help of `if-elif-else` we can make a tricky decision. The `elif` statement checks multiple conditions one by one and if the condition fulfills, then executes that code.

```
Syntax:
if condition-1:
     statement 1
elif condition-2:
     stetement 2
elif condition-3:
     stetement 3
     ...
else:
     statement
```

6. Nested if-else statement:

A nested `if` statement is an `if` statement inside another `if` or `else` block. It allows for more complex conditional logic.

the nested `if-else` statement is an if statement inside another `if-else` statement. It is allowed in Python to put any number of `if` statements in another `if` statement.

Indentation is the only way to differentiate the level of nesting. The nested `if-else` is useful when we want to make a series of decisions.

```
Syntax:
if condition1:
    # code block for condition1
    if condition2:
        # code block for condition2
    else:
        # code block for condition2 being false
else:
    # code block for condition1 being false
```

6. Short Hand If:

The short-hand `if` is a concise way to write an `if` statement in a single line.

```
Syntax:
result = value_if_true if condition else value_if_false
```

7. Short Hand If Else:

The short-hand `if-else` can be used for one-liner ternary operations.

```
Syntax:
result = value_if_true if condition else value_if_false
```